

M²Fabric: A Post-Quantum Cryptographic Multilayer Architecture for Hyperledger Fabric

Zara Laila Cheong¹, Ali Tufail¹, and Rosyzie Anna Apong¹

¹School of Digital Science, Universiti Brunei Darussalam, Bandar Seri Begawan, BE1410, Brunei Darussalam

Corresponding author: Zara Laila Cheong (email: zaralaila@icloud.com).

ABSTRACT Hyperledger Fabric relies on classical elliptic-curve cryptography across its entire transaction lifecycle, leaving every signing, certification, validation, and key-exchange operation vulnerable to a cryptographically relevant quantum computer. Prior post-quantum proposals each replaced only a subset of Fabric’s cryptographic layers, leaving the remainder quantum-exploitable. This article presents M²Fabric, a multilayer post-quantum architecture that simultaneously replaces all four live cryptographic layers of Hyperledger Fabric v2.5.12 and Fabric CA v1.5.12: the Blockchain Cryptographic Service Provider (BCCSP) signing layer, the certificate infrastructure, the Membership Service Provider identity-validation layer, and the Transport Layer Security (TLS) inter-node transport layer. The design uses NIST-standardised ML-DSA-44/65 (FIPS 204), FN-DSA-512 (FIPS 206 Initial Public Draft), and ML-KEM-768 (FIPS 203) through pure-Go libraries and is deployed in three migration phases. The principal engineering contribution is *buildPQCCchain()*, a certificate-chain validator that bypasses the Go standard library’s rejection of post-quantum object identifiers, enabling a live certificate authority to issue and validate post-quantum certificates at runtime. Verification spans NIST ACVP coverage, interface-compliance, round-trip, and end-to-end transaction execution with zero failures. An empirical evaluation on a two-organisation Caliper network ($n = 27$ runs per workload plus a six-run ECDSA baseline) shows the Phase 3 deployment reduces maximum sustainable write-heavy throughput from the 275.3 TPS ECDSA baseline by 3.5% for FN-DSA-512, 10.9% for ML-DSA-44, and 18.0% for ML-DSA-65, with bootstrap and Cohen’s d checks supporting the ordering. The results establish per-endorsement payload size, not signing speed, as the dominant throughput determinant.

INDEX TERMS Blockchain security, certificate chain validation, crypto-agility, Hyperledger Fabric, ML-DSA, ML-KEM, post-quantum cryptography, quantum-resistant key exchange.

I. INTRODUCTION

PRIOR post-quantum integrations of Hyperledger Fabric remain *partial*: each replaces only a subset of the platform’s cryptographic layers, leaving the remainder quantum-exploitable [1]–[5]. The recurring cause is a single implementation barrier — Go’s *crypto/x509* stack cannot parse or validate certificates carrying post-quantum object identifiers — which has confined prior work to *offline* certificate generation and left the live Fabric CA’s classical signing key active. Because trust in Fabric propagates from that CA, a migration that leaves it classical is not a transitional state but a structural vulnerability: a quantum adversary who recovers the classical root key can forge post-quantum certificates that even a post-quantum validator accepts. Migrating all four live cryptographic layers — BCCSP signing, X.509 certificate

issuance, MSP identity validation, and TLS transport — simultaneously, at runtime, and without modifying the Go standard library is therefore the open problem this article addresses.

We frame three research questions. **RQ1**: can all four live cryptographic layers of Fabric v2.5.x be migrated to NIST-standardised post-quantum algorithms simultaneously while preserving compatibility with the Fabric component ecosystem? **RQ2**: can a live CA issue, and a runtime MSP validate, post-quantum X.509 certificates without modifying Go’s *crypto/x509*? **RQ3**: what is the throughput, latency, and storage cost of the resulting deployment, and which algorithm should an operator choose?

The central artefact of M²Fabric is *buildPQCCchain()*, an OID-dispatched certificate-chain validator that resolves

the *crypto/x509* barrier of RQ2 and thereby makes a live post-quantum CA — and hence full four-layer coverage — attainable. Around it, M²Fabric provides a post-quantum BCCSP plugin (ML-DSA-44, ML-DSA-65, and FN-DSA-512 through constant-time pure-Go libraries), a Fabric CA extended to issue post-quantum X.509 certificates, and ML-KEM-768 hybrid key exchange at the TLS layer. Its contributions, to the best of our knowledge, are: (i) *buildPQCChain()*, the first OID-dispatched runtime validator to lift the Go standard-library X.509 barrier that had prevented live-CA integration in prior work [1]–[3]; (ii) the first runtime integration of all four primary cryptographic layers of Fabric v2.5.x under FIPS-standardised algorithms (FIPS 203, 204, and the FIPS 206 Initial Public Draft) in unmodified Go; (iii) an end-to-end performance characterisation ($n = 27$ runs per workload across nine algorithm-phase combinations, plus a six-run ECDSA baseline) establishing per-endorsement payload size, not signing speed, as the dominant throughput determinant; and (iv) resolution of the practical engineering barriers (post-quantum-OID certificate generation and chain validation, variable-length FN-DSA signatures, and CFSSL hook-based integration).

§II reviews related work, §III the methodology, and §IV the experimental setup and results; §V discusses completeness, generalisability, and limitations, and §VI concludes.

II. LITERATURE REVIEW

This section reviews post-quantum standardisation, post-quantum TLS performance, and prior Fabric integrations. The recurring gap is that each prior Fabric integration has replaced only a strict subset of the platform’s cryptographic layers, leaving the unaddressed layers quantum-exploitable irrespective of the strength of those replaced.

A. POST-QUANTUM CRYPTOGRAPHY AND STANDARDISATION

Shor’s algorithm renders the elliptic-curve discrete-logarithm problem and integer factorisation tractable, breaking ECDSA, ECDH, and RSA. NIST’s six-year standardisation [6] produced FIPS 203 (ML-KEM) [7], FIPS 204 (ML-DSA) [8], FIPS 205 (SLH-DSA) [9], and the FIPS 206 Initial Public Draft (FN-DSA) [10]. ML-DSA derives from CRYSTALS-Dilithium [11] and FN-DSA from Falcon [12], [13]. NIST guidance recommends hybrid constructions for transitional security [14], [15], instantiated for TLS by the IETF hybrid framework [16] and the *X25519MLKEM768* codepoint [17]. Ravi et al. [18] catalogue side-channel attacks against lattice schemes and confirm constant-time arithmetic is necessary but not sufficient for adversarial deployment [19].

B. POST-QUANTUM TLS

Paquin et al. [20] and Sikeridis et al. [21] characterised post-quantum TLS and found the principal cost is payload (larger KEM ciphertexts and signatures) rather than per-operation

computation, a theme that recurs at the application layer in the present work. The Open Quantum Safe *liboqs* [22] and Cloudflare CIRCL provide the reference implementations; CIRCL’s pure-Go ML-DSA and ML-KEM run in production at Cloudflare scale.

C. POST-QUANTUM HYPERLEDGER FABRIC

Hyperledger Fabric [23] is a permissioned blockchain whose transaction throughput under classical cryptography has been characterised by Baliga et al. [24], Kuzlu et al. [25], Nasir et al. [26], and Thakkar et al. [27]. These analyses identify endorsement-signature verification at the committing peer as a principal scaling bottleneck.

Prior post-quantum work on Hyperledger Fabric falls into four categories. *Algorithm benchmarks without Fabric integration* (Campbell [28], [29]; qTESLA was eliminated from NIST Round 3 [6]). *Attack-surface surveys* (Brotsis et al. [30]). *Application-layer integrations* place primitives in chaincode or enclaves while leaving Fabric’s BCCSP/CA/MSP/TLS classical: Lee [31] (ML-DSA, Vesper, TDUE under Intel SGX), Das et al. [32] (Kyber/Dilithium in genomic chaincode), and Revathi and Suganthi [33] (Kyber/Falcon/Dilithium in hospital chaincode). *Endorsement-policy proxies* configure multi-organisation signatures as governance rather than cryptographic replacement: Razdan and Nene [5] report 119% latency increase under dual-signature endorsement, with no cryptographic modification.

The four *infrastructure-layer integrations* (Kasula et al. [4], PQFabric (Holcomb et al.) [1], Abbas et al. [3], and Hu [2]) each address a strict subset of the four cryptographic layers. PQFabric used CGo bindings to *liboqs* with pre-FIPS parameters and established that per-payload overhead, not signing speed, dominates performance — an observation M²Fabric extends to FIPS-finalised algorithms. Abbas et al. added post-quantum certificate generation to the offline Cryptogen tool but did not address the live CA, TLS, or MSP chain validation; Hu targeted Fabric 3.0 but required a non-standard Go compiler fork.

Across all four categories, no prior integration overcomes the *runtime* certificate-validation barrier: Go’s *crypto/x509* cannot validate a chain carrying post-quantum object identifiers, which confined every infrastructure-layer effort to offline certificate generation and left the live CA’s classical signing key active — the gap that keeps the highest-severity threat (TS1) open regardless of what else is migrated. M²Fabric is, to the best of our knowledge, the first integration to overcome this barrier and address all four live cryptographic layers of the current Fabric LTS simultaneously using FIPS-standardised algorithms in unmodified Go. Table 1 consolidates the coverage. PQFabric and Hu use pre-FIPS lattice parameters; Kasula et al.’s primary SM2 algorithm is elliptic-curve-based and therefore equally vulnerable to Shor’s algorithm despite the “post-quantum” framing.

TABLE 1: Coverage of prior post-quantum Fabric work.

Work	Ver.	BCCSP	Cert.	MSP	TLS	Std Go
Campbell [28], [29]	—	—	—	—	—	—
Brotsis et al. [30]	—	—	—	—	—	—
Lee [31]	2.x	app.	—	—	—	part.
Das et al. [32]	2.x	app.	—	—	—	—
Revathi & Suganthi [33]	2.x	app.	—	—	—	—
Razdan & Nene [5]	2.x	prox.	—	—	—	—
Kasula et al. [4]	1.4	✓	—	—	—	part.
PQFabric [1]	1.4	✓	offl.	—	—	—
Abbas et al. [3]	2.x	✓	offl.	—	—	✓
Hu [2]	3.0	✓	offl.	—	—	—
M²Fabric	2.5.12	✓	✓	✓	✓	✓

app.: chaincode/enclave; prox.: endorsement-policy proxy; offl.: offline cert generation only.

III. METHODOLOGY

A. CONSTRUCTION METHODOLOGY

The implementation follows a design-science artefact-construction methodology. The target platform is Hyperledger Fabric v2.5.12 (commit *af0b647*) and Fabric CA v1.5.12. Cloudflare CIRCL v1.6.3 provides pure-Go ML-DSA-44/65 and ML-KEM-768; *go-fn-dsa* v0.2.0 provides FN-DSA-512. Go 1.24 is the minimum required version (it introduced *tls.X25519MLKEM768*, replacing the pre-standard *X25519Kyber768Draft00* from Go 1.23). The implementation is structured as non-invasive patches that preserve existing module structure, interface contracts, and test coverage by introducing post-quantum functionality through new files and types rather than modifying existing functions wherever feasible.

B. DESIGN REQUIREMENTS AND PRINCIPLES

The implementation is governed by eight formal integration requirements (R1–R8) derived from a component-level quantum threat analysis of Hyperledger Fabric. The analysis enumerates twenty-five quantum-vulnerable lifecycle operations (Op-01 through Op-25) and consolidates them into the five threat scenarios of Table 2. Three *functional* requirements close these scenarios — R1 (PQC signing at every signing point), R2 (PQC key encapsulation in TLS), and R3 (runtime PQC certificate issuance by the live CA) — supported by five *non-functional* requirements: R4 (preserve the BCCSP interface), R5 (crypto-agility via configuration), R6 (functional correctness), R7 (constant-time operations), and R8 (phased migration). TS1 and TS3 carry the joint-highest priority (Critical severity, High exploitability) because the root-CA and orderer signing keys are recoverable directly from public ledger content.

The most technically demanding requirement is R3: it requires bypassing Go’s standard-library X.509 certificate-

TABLE 2: Threat scenarios: the lifecycle operations each targets, its severity and exploitability, and the functional requirement that closes it.

TS	Scenario	Affected operations	Sev./Expl.	Closed by
TS1	Identity impersonation (cert. forgery)	Op-04, 16, 17, 18, 19, 21, 22	Crit./High	R3
TS2	Transaction-signature forgery	Op-01, 03, 05, 07, 09, 14	High/High	R1
TS3	Block integrity (orderer-key recovery)	Op-10, 12, 13, 15	Crit./High	R1
TS4	TLS session decryption (HNDL)	Op-02, 06, 08, 11, 13, 20, 24	Med./Med.	R2
TS5	Gossip-protocol manipulation	Op-23, 24	Med./Low	R2

generation API for post-quantum keys, making the *build-PQCCChain()* validator (§G) a direct consequence, since a live CA that issues post-quantum certificates requires a live validator that can verify them.

The requirements are jointly satisfied only under *full* coverage: migrating any strict subset of the four layers leaves at least one quantum-exploitable, so M²Fabric migrates all four — the property that distinguishes it from the partial integrations of prior work and that §A quantifies through the coverage identity (3).

C. POST-QUANTUM ALGORITHM SELECTION

With the design principles fixed, the next step is to select the algorithms that instantiate them. The selection is constrained by R1 to NIST-standardised algorithms from FIPS 203, 204, 205, or 206, and is then guided by four criteria: security-level appropriateness, anticipated blockchain performance, availability of production-quality pure-Go implementations, and compatibility with Go’s certificate and TLS infrastructure. The decisive criterion is the per-endorsement payload $\Pi(a) = |\sigma_a| + |\text{cert}_a|$ (signature plus DER-encoded certificate): because $\Pi(a)$ is paid on every endorsement in every block whereas signing latency is amortised across a block, payload rather than signing speed should govern the choice — a selection rule the empirical results (§E) confirm and quantify.

1) DIGITAL SIGNATURE ALGORITHM SELECTION

Three signature algorithms are evaluated. **ML-DSA** (FIPS 204) is based on module-lattice Module-LWE [11]: ML-DSA-44 provides NIST Category 2 (2,420-byte signatures) and ML-DSA-65 provides Category 3 (3,309 bytes), both deterministic and $33.6\times$ – $46.0\times$ larger than ECDSA P-256 (71–72 bytes). **FN-DSA-512** (FIPS 206 IPD) is based on NTRU/SIS lattices [12], [13] and provides Category 1 security with a compact variable-length Gaussian

signature of ≈ 666 bytes (809-byte maximum), roughly $3.6\times$ smaller than ML-DSA-44; the trade-off is signing $\approx 53\%$ slower than ML-DSA-44 (≈ 698 versus ≈ 458 μ s), a relationship that inverts when output size dominates, as in the blockchain context. **SLH-DSA** (FIPS 205) [9] offers hash-based algorithmic diversity but its 7,856–49,856-byte signatures would inflate ledger storage by $100\text{--}700\times$ over ECDSA and is excluded from the primary integration. Table 3 consolidates the size parameters.

TABLE 3: Digital signature algorithms evaluated for M²Fabric.

Algorithm	Standard	Security	Pub. Key	Priv. Key	Signature
ECDSA P-256	N/A	128-bit	65 B	32 B	71–72 B
ML-DSA-44	FIPS 204	Cat. 2	1312 B	2528 B	2420 B
ML-DSA-65	FIPS 204	Cat. 3	1952 B	4000 B	3309 B
FN-DSA-512	FIPS 206	Cat. 1	897 B	1281 B	666 B
SLH-DSA-128s	FIPS 205	Cat. 1	32 B	64 B	7856 B

The DER-encoded certificate carries three algorithm-scaling components: the holder’s SubjectPublicKeyInfo, the issuing CA’s signature over the TBS structure, and a fixed ~ 440 -byte RFC 5280 [34] envelope. Measured certificate sizes (M²Fabric Fabric CA, Fabric-realistic DNs) are 570, 1,945, 4,126, and 5,655 bytes for ECDSA P-256, FN-DSA-512, ML-DSA-44, and ML-DSA-65 respectively. Because a post-quantum CA signs each certificate with the same algorithm as the endorser, a single endorsement effectively pays for two post-quantum signatures plus one public key plus the envelope; under a single-endorser policy, a 500-transaction block therefore carries $500\Pi(a)$ of this payload over the chaincode read-write sets. The resulting per-endorsement payloads are $\Pi = 641$ bytes for ECDSA P-256 (71-byte signature plus 570-byte certificate), 2,611 bytes for FN-DSA-512 ($4.07\times$), 6,546 bytes for ML-DSA-44 ($10.21\times$), and 8,964 bytes for ML-DSA-65 ($13.98\times$) — an excess of 1,970, 5,905, and 8,323 bytes over the ECDSA baseline, which Figure 1 decomposes into the shared 641-byte baseline and this excess. The Falcon-derived FN-DSA-512 keeps both signature (≈ 666 bytes) and public key (897 bytes) compact, whereas the Dilithium-derived ML-DSA family trades larger outputs for faster signing, with ML-DSA-65 adding roughly 37% to the signature size over ML-DSA-44. These multipliers predict the throughput ordering of §E.

2) KEY ENCAPSULATION MECHANISM SELECTION

For TLS key exchange, ML-KEM-768 (FIPS 203, NIST Category 3) is selected [7]. Go’s *tls.X25519MLKEM768* named-curve constant combines X25519 classical ECDH with ML-KEM-768 without additional dependencies and is standardised by the IETF as a TLS 1.3 key-exchange group [16]. The choice of Category 3 for ML-KEM relative to Category 1 for FN-DSA-512 reflects asymmetric performance sensitivity: KEM is performed once per TLS session,

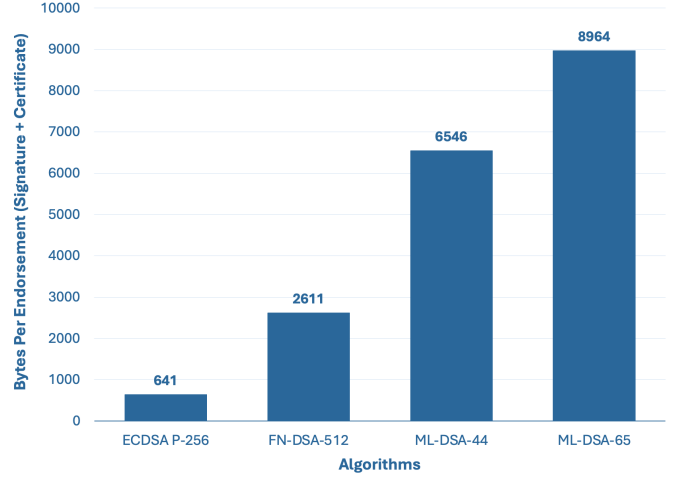


FIGURE 1: Per-endorsement ledger payload $\Pi(a)$ by algorithm; each bar splits the 641-byte ECDSA baseline from the post-quantum excess.

so the higher security level is negligible relative to the per-transaction signature cost.

D. MULTILAYER ARCHITECTURE

Figure 2 presents the M²Fabric four-layer architecture (purple: BCCSP signing; teal: CA/MSP identity; blue: TLS transport), showing for each layer its primary integration point, the library and standard it depends on, the phase in which it activates, and the threat scenarios it closes.

The BCCSP layer implements Fabric’s *bccsp.Key/bccsp.KeyGen* interfaces through a pure-Go PQC plugin (CIRCL for ML-DSA, *go-fn-dsa* for FN-DSA, no CGO). The Fabric CA layer implements the CFSSL *crypto.Signer* interface and constructs post-quantum X.509 certificates via *CreatePQCCertificate()*, bypassing *x509.CreateCertificate* through custom ASN.1 encoding. The MSP layer implements *buildPQCCChain()* in *misp/pqc.go*, dispatching chain validation on the child’s *signatureAlgorithm* OID and enforcing validity-period, *keyCertSign*, and AKID/SKID checks, bypassing *x509.Certificate.Verify*. The TLS layer sets *tls.X25519MLKEM768* as the preferred key-agreement identifier. Each layer is a self-contained set of source changes interacting with the others only through existing Fabric interface contracts: the certificate format produced by the CA is compatible with the MSP chain validator, embedded public keys are importable by the BCCSP plugin, and algorithm OIDs in the detection table match the certificate generator’s.

1) THREE-PHASE MIGRATION DESIGN

The implementation is structured in three phases that progressively activate the four layers, satisfying R8 and enabling per-layer empirical attribution (§D). The residual quantum-attack surface after phase ϕ is

LAYER	INTEGRATION POINT · LIBRARY · STANDARD	PHASE · THREAT SCENARIO
BCCSP Signing Layer	bccsp.Key · bccsp.KeyGen interfaces Cloudflare CIRCL v1.6.3 · go-fn-dsa v0.2.0 FIPS 204 / FIPS 206 · Cat. 1–3 · pure Go, no CGO	PHASE 1 TS2 · TS3
Fabric CA Certificate Authority	PQCSigner satisfies crypto.Signer buildPQCCert() · custom ASN.1 bypass FIPS 204/206 · CIRCL + go-fn-dsa · bypasses x509.CreateCertificate	PHASE 2 TS1
MSP Identity Authority	buildPQCChain() in msp/pqc.go OID dispatch · bypasses x509.Verify FIPS 204/206 · CIRCL + go-fn-dsa · endorsement + commit time	PHASE 2 TS1
TLS Inter-Node Layer	tls.X25519MLKEM768 in core/comm dual-CA: PQC CA → MSP · classical CA → TLS FIPS 203 · ML-KEM-768 · Go 1.23 stdlib · all inter-node sessions	PHASE 3 TS4 · TS5

FIGURE 2: Multilayer post-quantum architecture.

$$R(\phi) = \sum_{i=1}^5 s_i (1 - c_i(\phi)), \quad (1)$$

where $s_i > 0$ is the severity weight of threat scenario TS_i and $c_i(\phi) \in \{0, 1\}$ indicates whether TS_i is closed at the end of phase ϕ . The migration design drives $R(\phi)$ monotonically to zero: $R(0) = \sum_i s_i$ (classical baseline), $R(1) > 0$ (TS1 remains open), $R(2)$ closes TS1–TS3, and $R(3) = 0$ once the hybrid-TLS layer closes TS4 and the TLS component of TS5. Figure 3 presents this sequence as a timeline; the red Phase 1 badge marks the open TS1 surface that motivates the multilayer design.

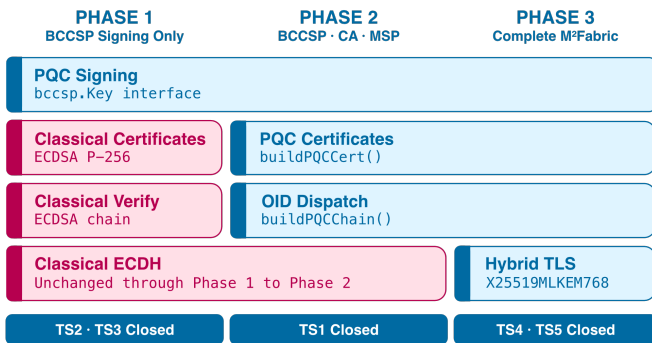


FIGURE 3: Three-phase migration sequence.

Phase 1 activates the PQC BCCSP signing layer only, partially addressing TS2 and TS3 but leaving TS1 entirely open because the CA’s ECDSA keys remain harvest targets.

Phase 2 adds the PQC certificate infrastructure: all enrolled identities receive post-quantum certificates, MSP

validation uses *buildPQCChain()*, and TS1–TS3 are fully addressed. This is the most operationally significant step, requiring re-enrolment of all nodes, clients, and administrators under the M²Fabric extension.

Phase 3 adds hybrid TLS key exchange (*X25519MLKEM768*) for all inter-node connections, closing TS4 and the TLS component of TS5 and yielding $R(3) = 0$.

E. BCCSP LAYER IMPLEMENTATION

The Blockchain Cryptographic Service Provider interface, defined in *bccsp/bccsp.go*, is Fabric’s abstraction for cryptographic services; no component contains algorithm-specific code outside the BCCSP. The M²Fabric plugin is grafted into the software backend through a self-contained PQC wrapper (*bccsp/sw/pqkey.go*) and a factory (*bccsp/factory/pqcfactory.go*) that registers the PQC backend without disturbing classical call sites. The plugin implements the full *bccsp.BCCSP* interface for ML-DSA-44, ML-DSA-65, and FN-DSA-512, instantiated when *core.yaml* specifies *Default: PQC*. Key generation invokes CIRCL or *go-fn-dsa* with *crypto/rand* and stores keys in DER format. ML-DSA signing uses the pure FIPS 204 mode (full-message with internal SHAKE-256); FN-DSA signing produces a variable-length NTRU-lattice signature centred on ~666 bytes, which Fabric’s unbounded protobuf signature fields accommodate without serialisation changes. *KeyImport* inspects the SPKI OID and either deserialises post-quantum bytes or delegates classical OIDs to the software BCCSP; *Hash* and *GetHashOpts* are delegated unconditionally since SHA-256 and SHA-3 are not replaced.

All Fabric signing paths (peer endorsement, orderer block signing, client proposal signing) call *Sign* through existing helpers, so switching *Default* from software to *PQC* redirects all signing and verification. Algorithm dispatch is table-driven (a map keyed by algorithm name): adding a new algorithm requires only a new entry, not new code paths. Private keys are serialised with their algorithm identifier and type-checked at load to prevent cross-algorithm misuse. The integration spans approximately 1,550 new or modified implementation lines of Go (excluding test code), distributed across the BCCSP layer (*bccsp/sw/pqckey.go* and *bccsp/factory/pqcfactory.go*, 296 LOC), the MSP layer (*misp/pqc.go* validator, 358 LOC, with a 14-LOC dispatch hook in *misp/mispimpl.go*), nine inter-module TLS delta-lines, the Fabric CA certificate-issuance path (*fabric-ca/util/pqcsigner.go*, 758 LOC), and configuration glue. The external pure-Go *pqc-bccsp* module provides the algorithm primitives shared by both the signing backend and the chain validator’s dispatch table.

F. CERTIFICATE INFRASTRUCTURE IMPLEMENTATION

The central implementation challenge is the incompatibility between post-quantum OIDs and Go’s *crypto/x509* package, which (through Go 1.24, the version targeted by this work) supports only RSA and ECDSA signature algorithms. Calling *x509.CreateCertificate* with a post-quantum key, or *x509.Certificate.Verify* on a post-quantum signature, returns an unsupported-algorithm error — the barrier that, as noted in §II, confined prior work to offline generation and left the TS1 gap open. M²Fabric resolves it through a custom certificate construction function in Fabric CA and the custom *buildPQCCChain()* validator in the MSP.

1) FABRIC CA POST-QUANTUM EXTENSION

The M²Fabric extension adds post-quantum issuance to a single Fabric CA binary through three coordinated changes. *CreatePQCCertificate* constructs an X.509 v3 certificate manually in ASN.1 DER per RFC 5280 [34], placing the post-quantum public key under the NIST OID for ML-DSA-44/65 and the provisional OQS OID *1.3.9999.3.6* for FN-DSA-512 (to be superseded by the finalised FIPS 206 OID via a single-constant change). Integration with the signing path is achieved by registering a hook with the forked CFSSL local signer at package initialisation: when the certificate template’s public key is a *PQCPublicKey* (including the raw form populated from a client-submitted PQC CSR), the hook dispatches to *CreatePQCCertificate* and signs the TBS directly through the BCCSP plugin with no caller-side pre-hash (ML-DSA and FN-DSA perform the FIPS-mandated internal hashing). The enrolment handler contains a complementary fallback: when *x509.ParseCertificateRequest* fails on an unknown post-quantum OID, *util.ExtractCNFromCSR* extracts the subject common name directly from the ASN.1 bytes. The resulting binary remains a drop-in replacement for the standard Fabric CA with unchanged API and endpoint surface.

The MSP identity certificate is the artefact Fabric authenticates against in every transaction, endorsement-policy evaluation, and channel configuration update; the certificate-infrastructure layer is therefore fully post-quantum end-to-end, from CA root self-signing through runtime chain validation at the committing peer. The IETF-track composite-signature construction [35] provides a future extensibility path that the M²Fabric dispatch table and *CreatePQCCertificate* generalise to once it standardises.

G. MSP IDENTITY VALIDATION: THE *buildPQCCChain()* VALIDATOR

MSP identity validation verifies certificate chains for every identity in channel transactions. The standard MSP uses *x509.Certificate.Verify*, which rejects post-quantum chains, so the post-quantum lifecycle requires a dedicated validator. Figure 4 shows the *buildPQCCChain()* control flow. The validator walks the chain from end-entity to root under a bounded depth of ten and terminates successfully when it reaches a self-signed certificate matching a configured MSP root anchor. At each non-terminal link, *findIssuerFor* locates the issuer by matching *RawIssuer* against intermediate-CA and root-CA Subject DN’s, then *verifyChainLink* runs the per-link checks. Algorithm dispatch is keyed on both the child’s *signatureAlgorithm* OID and the issuer’s SPKI OID: classical pairs route to *Certificate.CheckSignatureFrom*, post-quantum pairs invoke CIRCL (ML-DSA) or *go-fn-dsa* (FN-DSA) on the *RawTBSCertificate* with no caller-side pre-hash. A legacy-format fallback re-resolves dispatch when the child carries a classical placeholder OID but the issuer’s SPKI is post-quantum, and any cross-algorithm inconsistency is rejected; a SHA-256 pre-hash retry preserves verifiability of pre-fix-generator credentials. Hybrid chains mixing classical and post-quantum links are supported to accommodate phased migrations.

verifyChainLink applies five structural checks in order before signature verification: issuer validity window; child validity window (skipped at self-signed roots); *BasicConstraints.IsCA*; *keyCertSign* when *KeyUsage* is set; and AKID/SKID matching when both are present per RFC 5280 [34].

The OID dispatch is what makes runtime validation possible where Go’s own path fails. *x509.Certificate.Verify* selects its verification routine from a fixed table of RSA/ECDSA identifiers and rejects an unrecognised OID *before* signature checking, so a post-quantum chain never reaches verification. *buildPQCCChain()* instead keys dispatch on the issuer’s *SubjectPublicKeyInfo* OID — the one field that authoritatively records which primitive signed the child — and routes each link to CIRCL, *go-fn-dsa*, or Go’s classical path accordingly. It is invoked from *getUniqueValidationChain* only when a post-quantum certificate is present; classical-only deployments take the unchanged Go path. The chain walk is $O(d)$ in chain depth d (bounded at ten), performing one issuer lookup and one signature verification per link, so

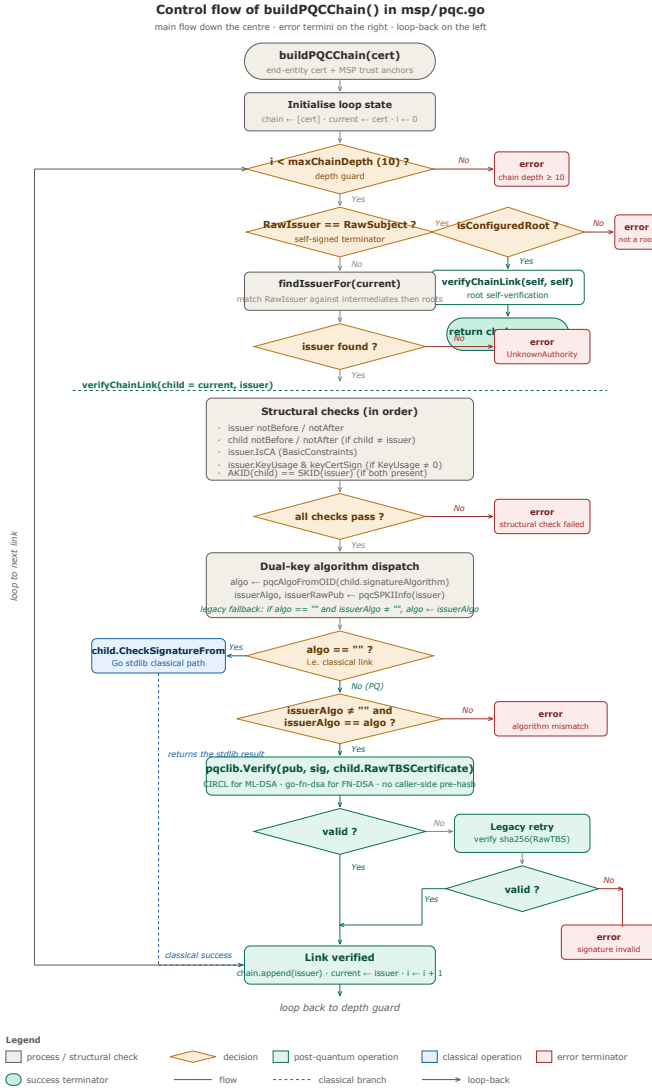


FIGURE 4: Control flow of buildPQCChain().

the marginal cost over the classical validator is a single post-quantum verification per identity per block at Op-16 — the operation §E identifies as the Phase-2 bottleneck.

H. TLS LAYER IMPLEMENTATION

Go 1.24's *crypto/tls* exposes *X25519MLKEM768* as a *tls.CurveID* constant, performing both X25519 ECDH and ML-KEM-768 encapsulation in the ClientHello and deriving the session key

$$K_{\text{sess}} = \text{HKDF}(K_{\text{X25519}} \| K_{\text{MLKEM}}), \quad (2)$$

which is secure provided either contributing exchange is secure [14]–[17]. M²Fabric sets *CurvePreferences* to *[tls.X25519MLKEM768, tls.X25519, tls.CurveP256]*, placing the hybrid first with classical fallbacks; Raft inter-orderer mutual TLS uses the same configuration. The hybrid exchange closes TS4 and the TLS component of TS5 independently of the certificate-infrastructure layer.

IV. EXPERIMENTAL SETUP AND RESULTS

A. TEST NETWORK CONFIGURATION

The test network is two organisations (one peer each) and a single Raft orderer on a single physical host. The same M²Fabric binary is used for all phases: Phase 1 enables the PQC BCCSP with bootstrap ECDSA certificates from a modified Cryptogen; Phase 2 adds Fabric CA with the post-quantum extensions and re-enrols all identities to post-quantum certificates; Phase 3 adds *X25519MLKEM768* to the TLS preference list and restarts all nodes. The standard Fabric asset-transfer-basic chaincode is deployed without modification. The topology exercises the full transaction lifecycle, the endorsement-policy mechanism (both organisations signing), and the Raft consensus path.

B. BENCHMARK METHODOLOGY

The benchmark runs on a single host (Intel Core i9-9980HK, 8 cores at 2.4–5.0 GHz, 64 GB DDR4, macOS, Docker) executing Hyperledger Fabric v2.5.12 (commit *af0b647*), Fabric CA v1.5.12, Hyperledger Caliper v0.6.0, Go 1.24, Cloudflare CIRCL v1.6.3, and *go-fn-dsa* v0.2.0. Channel configuration uses a 500-transaction block, 2-second timeout, and any-1-of-2 endorsement policy executing the standard asset-transfer chaincode. Caliper [36] sweeps thirteen send rates (25–1,000 TPS) for three independent runs in both write-heavy (90%) and mixed (50%) profiles, with a clean genesis reset before each write-heavy run.

Maximum sustainable throughput Γ_{max} is the mean achieved throughput over the plateau region — the contiguous range of send rates across which achieved throughput no longer rises with the target rate. Four metrics are reported: Γ_{max} (TPS), saturation latency, ledger storage (MB/peer, measured directly after each run), and block propagation rate (blocks/s from Prometheus over a 30-minute window).

C. VERIFICATION AND CORRECTNESS ASSURANCE

ML-DSA primitive correctness inherits from Cloudflare CIRCL's upstream NIST ACVP coverage for FIPS 204; FN-DSA correctness is verified through round-trip integration tests on real X.509 *TBSCertificate* fixtures since ACVP vectors for *go-fn-dsa* v0.2.0 are not yet available. The plugin satisfies the *Sign/Verify/KeyImport/GetKey* contracts through unmodified call sites in peer, orderer, and client SDK; the table-driven test suite sweeps all three PQ algorithms and confirms error-path semantics (tampered signatures, expired certificates, unknown issuers, untrusted self-signed). End-to-end execution covers the five lifecycle stages in each of the three deployment phases for all three algorithms with zero failures. With $n = 27$ benchmark runs per workload and $\approx 1.7 \times 10^5$ committed post-quantum transactions, the Clopper-Pearson 95% upper bound on the per-transaction failure probability is $p < 1.8 \times 10^{-5}$.

Four quantitative properties are established: (i) integration coverage

$$\delta_{M^2Fabric} = \frac{|\{\text{post-quantum deployed}\}|}{|\{\text{ECDLP-vulnerable identified}\}|} = \frac{4}{4} = 1, \quad (3)$$

with no prior published integration achieving $\delta = 1$ [1]–[3]; (ii) signature-size amplification of $9.3\times$, $33.6\times$, and $46.0\times$ for FN-DSA-512, ML-DSA-44, and ML-DSA-65 relative to 72-byte ECDSA; (iii) hybrid TLS session-key security from (2), secure as long as either contributing exchange is secure; and (iv) zero verification failures across all ACVP, round-trip, interface, and end-to-end tests.

D. BASELINE AND PER-PHASE THROUGHPUT RESULTS

1) ECDSA P-256 BASELINE

The ECDSA P-256 baseline achieves **275.3 TPS** (standard deviation 1.35) write-heavy with 0.18-s plateau latency and 1,690 MB per-peer ledger, and **344.2 TPS** (standard deviation 1.93) mixed with 0.11-s latency and 3,266 MB per-peer ledger. The saturation knee sits at 200–300 TPS write-heavy and 300–400 TPS mixed. These figures are consistent with prior characterisations of standard Fabric [23]–[26] after accounting for hardware and topology differences.

2) PHASE 1: BCCSP POST-QUANTUM SIGNING

Phase 1 replaces ECDSA signing at the BCCSP layer while leaving certificates and TLS unchanged, isolating the signature-algorithm impact. All three algorithms follow the same sub-knee saturation profile as ECDSA; above the knee each plateau sits below the ECDSA ceiling in proportion to signature size, with FN-DSA-512 incurring the smallest reduction (-1.7% both workloads), then ML-DSA-44 (-4.2% write, -1.4% mixed) and ML-DSA-65 (-3.5% write, -4.1% mixed) comparable at this phase — the write-heavy ML-DSA-65/ML-DSA-44 gap lies within ML-DSA-65’s standard deviation (6.55 TPS, the study’s largest). Plateau latencies are 0.18–0.20 s, indistinguishable from baseline. Per-phase numbers are consolidated in Table 4.

3) PHASE 2: POST-QUANTUM CERTIFICATE INFRASTRUCTURE

Phase 2 adds post-quantum X.509 infrastructure: the CA issues post-quantum-signed certificates and *buildPQCChain()* validates them at Op-16 (the committing-peer chain revalidation, run for every endorsement signature in every block). The certificate layer introduces substantially larger reductions than Phase 1 for the ML-DSA family: FN-DSA-512 declines by 3.3%/3.2% (write/mixed) from the 1.7% Phase 1 penalty, reflecting per-endorsement SPKI cost; ML-DSA-44 by 9.9%/8.9%; and ML-DSA-65 by 18.1%/13.0%, the largest single-phase impact measured.

The dominant Phase 2 effect is not throughput reduction but certificate-processing latency (Figure 5). Phase 1 latencies are indistinguishable from the 0.18-s ECDSA baseline; the Phase 2 transition produces a qualitative jump to 1.71 s for FN-DSA-512 ($9.5\times$), 5.47 s for ML-DSA-44 ($30\times$), and 10.20 s for ML-DSA-65 ($57\times$). This reflects the Op-16

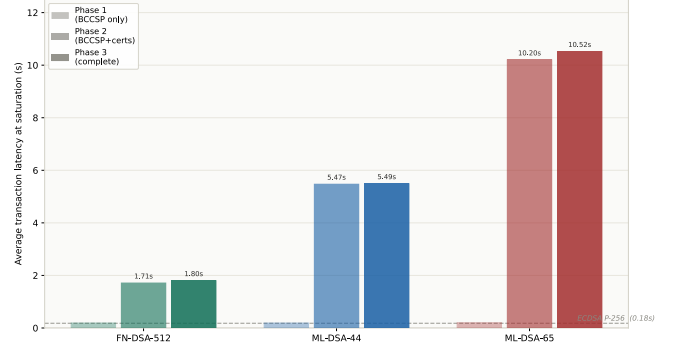


FIGURE 5: Saturation latency by algorithm and phase.

bottleneck [27]: a 500-transaction write-heavy block carries up to 500 chain validations, each over an SPKI of 897, 1,312, or 1,952 bytes respectively (versus 65 for ECDSA). Mixed-workload latencies are far lower (0.14, 0.18, 0.77 s) because fewer write transactions per unit time mean fewer certificates per block.

4) PHASE 3: COMPLETE M²FABRIC DEPLOYMENT

Phase 3 activates hybrid TLS (*X25519MLKEM768*) for all inter-node connections, completing the deployment. FN-DSA-512 achieves 265.5 TPS write-heavy and 332.7 TPS mixed (-3.5% , -3.3%); ML-DSA-44 achieves 245.2 and 317.0 TPS (-10.9% , -7.9%); and ML-DSA-65 achieves 225.8 and 301.1 TPS (-18.0% , -12.5%). Phase 2-to-Phase 3 changes are small, span zero, and lie within each Phase 3 standard-deviation envelope: hybrid TLS contributes a per-transaction overhead below the detection threshold of the $n = 3$ design, as expected when connection-level key-exchange cost amortises across many transactions per TLS session.

E. CONSOLIDATED ANALYSIS AND PERFORMANCE MODELS

The central finding of the evaluation is that per-endorsement payload size, not signing speed, determines throughput. Figures 6 and 7 trace the per-phase throughput for each algorithm and Table 4 attributes the reductions: for FN-DSA-512 the 3.5% write-heavy total splits roughly evenly between Phases 1 and 2, whereas for ML-DSA-44 and ML-DSA-65 the certificate layer dominates (5.7 and 14.6 percentage points respectively) and Phase 3 TLS contributes under 0.2 percentage points in every case. Every reduction tracks the per-endorsement payload (2,611/6,546/8,964 bytes for FN-DSA-512/ML-DSA-44/ML-DSA-65 versus 641 for ECDSA P-256), applied to every endorsement in every block — independently corroborating the PQFabric finding [1]. The attribution table reports absolute TPS together with the contribution of each phase to the total reduction from the ECDSA P-256 baseline; the Phase 3 column reports the change from Phase 2 to Phase 3, which isolates the hybrid TLS overhead.

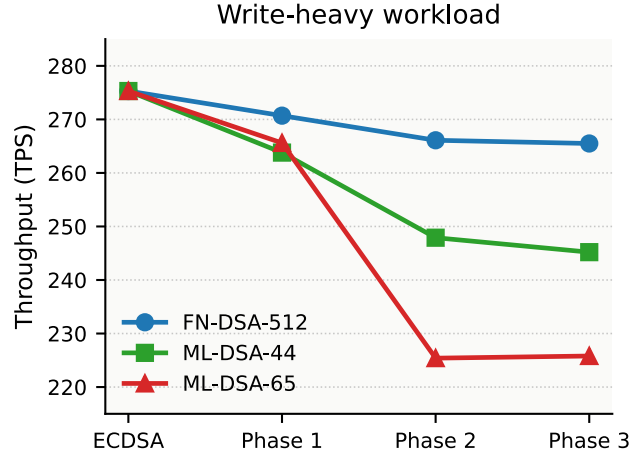


FIGURE 6: Phase-by-phase throughput progression, write-heavy workload.

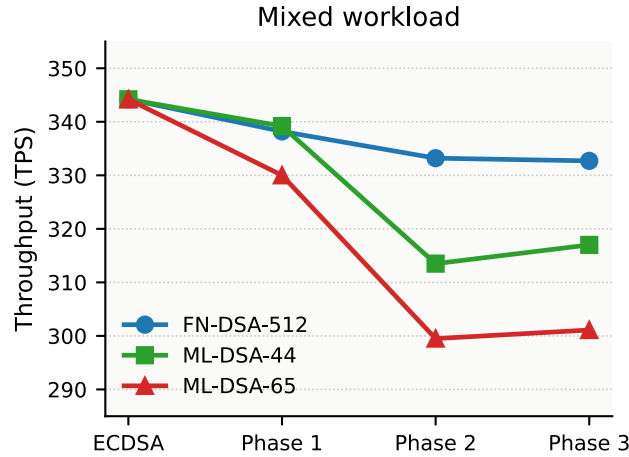


FIGURE 7: Phase-by-phase throughput progression, mixed workload.

TABLE 4: Phase-by-phase throughput attribution.

Algorithm	Workload	ECDSA	Phase 1	Phase 2	Phase 3	Total Δ
FN-DSA-512	Write	275.3	270.7 (−1.7%)	266.1 (−1.6pp)	265.5 (−0.2pp)	−3.5%
	Mixed	344.2	338.2 (−1.7%)	333.2 (−1.5pp)	332.7 (−0.2pp)	−3.3%
ML-DSA-44	Write	275.3	263.8 (−4.2%)	247.9 (−5.7pp)	245.2 (−1.0pp)	−10.9%
	Mixed	344.2	339.2 (−1.4%)	313.5 (−7.5pp)	317.0 (+1.1pp)	−7.9%
ML-DSA-65	Write	275.3	265.6 (−3.5%)	225.4 (−14.6pp)	225.8 (+0.2pp)	−18.0%
	Mixed	344.2	330.0 (−4.1%)	299.5 (−8.9pp)	301.1 (+0.5pp)	−12.5%

1) THROUGHPUT DEGRADATION, LATENCY, AND STORAGE MODELS

Throughput means are reported with 95% Student- t intervals ($t_{0.025,2} = 4.303$); adjacent-algorithm intervals are non-overlapping for the headline write-heavy comparisons. The ordering $\text{FN-DSA-512} > \text{ML-DSA-44} > \text{ML-DSA-65}$ is robust to a non-parametric bootstrap (10^4 resamples) and

to very large effect sizes (Cohen’s $d \approx 18, 12$). Only small-effect comparisons (notably hybrid TLS) are underpowered; $n \geq 5$ with ANOVA is recommended for those. Phase 2 write latencies follow from the Op-16 certificate-validation bottleneck: empirically $\tau_{\text{cert}} \propto |pk|^\beta$ with $\beta \approx 2.30$, so doubling the public-key size quintuples per-certificate validation time.

Ledger storage (Table 5) confirms signature size as a second-order contributor: all three post-quantum algorithms produce write-heavy ledgers within 4% of the 1,690 MB ECDSA baseline, with per-write on-disk overhead of 1,706–1,756 bytes across all four algorithms despite signatures ranging from 72 to 3,309 bytes. The mechanism is Fabric’s block-level identity model: endorsement certificates are stored by reference in the peer identity store rather than embedded in every block, so signature size affects transmission and verification cost but not ledger footprint. For the mixed workload the physically meaningful metric is per-run ledger growth (the increment added by one benchmark run), which isolates a single run’s contribution independently of the network’s prior state; the three post-quantum values agree to within two percent and the $0.74\text{--}0.75\times$ multipliers relative to the ECDSA baseline quantify a throughput-mediated reduction (fewer committed transactions per unit time), not a per-transaction storage saving.

TABLE 5: Ledger storage: write-heavy 3-run mean and mixed per-run growth.

Algorithm	Write-heavy (3-run mean)		Mixed (per-run growth)	
	MB	Ratio	MB/run	Ratio
ECDSA P-256	1,690	ref.	1,875	ref.
FN-DSA-512	1,669	$0.99\times$	1,408	$0.75\times$
ML-DSA-44	1,633	$0.97\times$	1,410	$0.75\times$
ML-DSA-65	1,679	$0.99\times$	1,386	$0.74\times$

Block-propagation metrics from sustained-load Prometheus measurements isolate the bottleneck location: all post-quantum configurations achieve 17–18 blocks/s write-heavy and 9.5–9.8 blocks/s mixed, with average block processing time uniformly $\sim 1.0\text{--}1.1$ ms. The ~ 1 ms per-block figure confirms that Phase 2/3 reductions are driven by certificate-validation overhead at the committing peer, not by gossip propagation or orderer latency.

F. ALGORITHM SELECTION GUIDANCE

The phase attribution confirms the selection rule of §C: per-endorsement payload size, not signing speed, is the dominant determinant, since FN-DSA-512 delivers the highest end-to-end throughput despite signing $\sim 53\%$ more slowly than ML-DSA-44. The Phase 3 figures of Table 4 translate directly into deployment guidance along the throughput–latency trade-off: *FN-DSA-512* (Category 1) is the primary choice for throughput-sensitive deployments and early-transition

pilots; *ML-DSA-44* (Category 2) is the balanced option and NIST’s first-choice signature; and *ML-DSA-65* (Category 3) is reserved for high-assurance deployments where Category 3 is regulatorily mandated, at a certificate-validation latency of $57\times$ the ECDSA baseline.

G. SECURITY ANALYSIS OF THE IMPLEMENTATION

The security rests on three foundations: the security of the post-quantum algorithms (established by NIST’s six-year standardisation process [6]; no polynomial-time quantum algorithm is known for the underlying Module-LWE or NTRU/SIS problems), the correctness of the implementation (established by the verification of §C), and the integrity of the integration design, which preserves the existing Fabric security model: the BCCSP contract is satisfied exactly, MSP policy evaluation is unchanged, the TLS configuration is a subset of Go’s option space, and CA-issued certificates follow the RFC 5280 structure.

The custom code paths added above the primitive layer are not covered by upstream review; Table 6 identifies their residual attack surface, the mitigation already in place, and the risk that remains.

TABLE 6: Residual attack surface of the custom code paths: mitigation in place and remaining risk.

Concern	Mitigation in place	Residual risk
Malformed certificates	ASN.1 parser rejects failures by error-return, not panic	Robustness unproven without coverage-guided fuzzing
Algorithm-OID spoofing	Dispatch enforces issuer-SPKI/child consistency; classical-for-PQ substitution fails	Known-OID, wrong-length key bytes untested under adversarial corpora
Downgrade path	Legacy SHA-256-prehash fallback isolated to one retry	Fallback should be disabled post-cutover via a strict-mode toggle
Transport endpoint	Layer scoped to Fabric’s MSP identity model	TLS endpoint infrastructure outside the Fabric crypto model; not addressed

A four-stage adversarial evaluation is scheduled as immediate follow-up: *go-fuzz* coverage-guided fuzzing of the *buildPQCCChain()* ASN.1 entry points; an OID-spoofing corpus exercising truncated/oversized SPKI payloads, cross-algorithm OID/key mismatches, and AKID/SKID-disjoint chains; downgrade-resistance testing with the legacy SHA-256-prehash fallback disabled; and constant-time differential analysis on the signing and verification paths to detect regressions in the upstream CIRCL and *go-fn-dsa* constant-time guarantees. The implementation’s security stance is therefore

provisional pending this workstream and the limitations consolidated in §C.

V. DISCUSSION

This section consolidates the implications of the results of §IV along four dimensions: the completeness of the multilayer approach relative to prior partial integrations, the amplification of the harvest-now-decrypt-later threat by ledger immutability, the generalisability of the architecture to other permissioned blockchain platforms, and the alignment of the design with emerging national and international post-quantum migration strategies.

A. COMPLETENESS OF THE MULTILAYER APPROACH

The coverage identity (3) gives $\delta_{M^2Fabric} = 4/4 = 1$: every ECDLP-vulnerable layer is migrated. Any integration that leaves a strict subset classical retains a positive residual surface $R(\phi)$ (1) — a BCCSP-only migration, the shape of several prior proposals, leaves TS1 fully open, because a quantum adversary who recovers the classical CA key can mint *post-quantum* certificates that the post-quantum MSP accepts. Partial migration is therefore *a security anti-pattern, not a transitional state*: Phase 1 alone pays most of the throughput cost for no protection against the highest-severity threat. This extends the PQFabric [1] payload observation to the FIPS-finalised generation, while the pure-Go choice distinguishes M²Fabric from CGo-bound prior work [22].

A second implication is that permissioned blockchains amplify the harvest-now-decrypt-later threat structurally: the append-only ledger guarantees that the harvest set $\mathcal{H}(t)$ is non-decreasing, so every ECDSA-signed transaction, endorsement, block header, and certificate committed before migration remains a permanent harvest target. The coverage identity (3) addresses *future* cryptographic material only; at the 275.3 TPS write-heavy plateau, each day of continued ECDSA operation accumulates $\sim 2.4 \times 10^7$ key-equivalents into $\mathcal{H}$, so migration urgency is proportional to both the residual security lifetime Z in Mosca’s inequality [37] and the rate at which the unprotected ledger continues to accumulate harvestable material.

B. GENERALISABILITY TO OTHER PERMISSIONED BLOCKCHAINS

The four-layer decomposition is not Fabric-specific: the same operational categories recur in Quorum, Besu, Corda, and most enterprise-grade permissioned platforms. The most transferable artefact is the *buildPQCCChain()* pattern: any Go-based blockchain that imports *crypto/x509* is currently subject to the same post-quantum OID rejection, and the same dispatch-on-*signatureAlgorithm* structure resolves it. Two qualifications apply: FN-DSA-512’s OID is provisionally assigned within the OQS experimental arc *1.3.9999.3.x* until FIPS 206 is finalised (requiring a transition-window dispatch in any portable validator), and hybrid TLS support

outside Go 1.24+ requires either analogous stdlib support or in-process key-exchange code.

The design also aligns with three concurrently developing migration frameworks: the NIST transition roadmap [38] (RSA/ECC deprecation by 2030, disallowance by 2035), the NSA Commercial National Security Algorithm Suite 2.0 [39] (full adoption by 2035), and emerging national strategies such as the draft Brunei Post-Quantum Cryptography Migration Framework [40]. M²Fabric’s three-phase migration sequences these frameworks’ Validate and Deploy stages for the Fabric platform specifically, supplying a concrete *how* for the permissioned-blockchain layer of a national migration whose policy frameworks otherwise specify only the *what* and *when*.

C. LIMITATIONS AND THREATS TO VALIDITY

Four limitations bound the results. (i) *Single-host, single-workload benchmark*: all measurements come from one physical host running a two-organisation network with a single chaincode, endorsement policy, and block size. Multi-host, multi-organisation, and WAN behaviour is not evaluated; a multi-host re-run with $n \geq 5$ and ANOVA remains future work. Each post-quantum cell averages $n = 3$ runs ($n = 27$ per workload across the nine algorithm-phase cells), and the ECDSA baseline comprises $n = 6$. (ii) *Functional rather than adversarial verification*: the fuzz-testing, adversarial-corpus, and downgrade-resistance gaps are addressed by the planned four-stage workstream (§G). (iii) *Provisional FN-DSA-512 OID*: the transition to the finalised FIPS 206 OID is a single-constant change but requires a transition-window dispatch entry. (iv) *Side-channel scope*: primitive-level constant-time guarantees inherit from CIRCL and *go-fn-dsa*; HSM deployment and EM/fault-injection characterisation [18] remain open. External validity is bounded by the single chaincode workload; construct validity rests on the plateau-mean operationalisation of maximum sustainable throughput.

VI. CONCLUSION

This article presented M²Fabric, a runtime integration replacing all four live cryptographic layers of the current Hyperledger Fabric LTS with FIPS-standardised post-quantum algorithms in unmodified Go. Its enabling artefact is *buildPQCCChain()*, the OID-dispatched validator that lifts the Go *crypto/x509* barrier and makes a live post-quantum CA possible; the empirical evaluation then answers RQ3, showing per-endorsement payload size — not signing speed — as the dominant throughput determinant, with the certificate layer driving the cost and hybrid TLS effectively free per transaction.

Three implications follow: partial migration is a security anti-pattern, since an operator deploying only the signing layer pays most of the throughput cost for no protection against TS1; the dispatch-on-*signatureAlgorithm* pattern transfers to any Go-based permissioned blockchain that im-

ports *crypto/x509*; and the design aligns with the deprecation timelines of NIST IR 8547, the NSA CNSA 2.0 suite, and emerging national migration frameworks. The follow-up agenda comprises a multi-host benchmark at $n \geq 5$ with ANOVA, a CPU/latency-breakdown profile localising the Op-16 cost, the four-stage adversarial-evaluation workstream, and HSM characterisation. By removing the runtime certificate-validation barrier that Go’s *crypto/x509* imposed, M²Fabric shows that complete post-quantum migration of an enterprise permissioned blockchain is now technically feasible at modest and well-characterised cost.

Artifact Availability

The M²Fabric source (the post-quantum BCCSP plugin, the *buildPQCCChain()* validator, and the Fabric CA extension as patches against Fabric v2.5.12 commit *af0b647* and Fabric CA v1.5.12), together with the Caliper benchmark configurations, the network topology, and the raw measurement data underlying every figure and table, are openly available in the project repository (*[repository URL to be inserted on publication]*) to support independent reproduction.

REFERENCES

- [1] A. Holcomb, G. Pereira, B. Das, and M. Mosca, “PQFabric: A permissioned blockchain secure from both classical and quantum attacks,” in *Proceedings of the IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*, pp. 1–9, IEEE, 2021. arXiv:2010.06571.
- [2] J.-W. Hu, “A prototype for post-quantum cryptographic migration in hyperledger fabric,” in *Proceedings of the IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*, pp. 1–2, IEEE, 2025.
- [3] S. Abbas, A. Sultana, and G. Kaddoum, “Quantum-safe blockchain in hyperledger fabric,” *IEEE Networking Letters*, vol. 7, no. 1, pp. 61–65, 2025.
- [4] V. K. Kasula, S. B. Rakki, and R. Banoth, “Enhancing hyperledger fabric security with lightweight post-quantum cryptography and national cryptographic algorithms,” in *Proceedings of the 37th Conference of the FRUCT Association*, pp. 93–99, 2025.
- [5] R. Razdan and M. J. Nene, “Post quantum signature for blockchain,” in *2025 1st International Conference on Advancement in Futuristic Technologies (ICAFT)*, (Belagavi, India), pp. 1–5, IEEE, 2025.
- [6] G. Alagic, D. Apon, D. Cooper, Q. Dang, T. Dang, J. Kelsey, J. Lichtinger, Y.-K. Liu, C. Miller, D. Moody, R. Peralta, R. Perlner, A. Robinson, and D. Smith-Tone, “Status report on the third round of the NIST post-quantum cryptography standardization process,” NIST Interagency Report 8413, National Institute of Standards and Technology, Gaithersburg, MD, 2022.
- [7] National Institute of Standards and Technology, “Module-lattice-based key-encapsulation mechanism standard,” Federal Information Processing Standard 203, National Institute of Standards and Technology, Gaithersburg, MD, 2024.
- [8] National Institute of Standards and Technology, “Module-lattice-based digital signature standard,” Federal Information Processing Standard 204, National Institute of Standards and Technology, Gaithersburg, MD, 2024.
- [9] National Institute of Standards and Technology, “Stateless hash-based digital signature standard,” Federal Information Processing Standard 205, National Institute of Standards and Technology, Gaithersburg, MD, 2024.
- [10] National Institute of Standards and Technology, “FN-DSA: FALCON-based digital signature standard (initial public draft),” Federal Information Processing Standard 206 (IPD), National Institute of Standards and Technology, Gaithersburg, MD, August 2025. Initial Public Draft, submitted for approval 28 August 2025; final standard expected late 2026 or early 2027.

- [11] L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, P. Schwabe, G. Seiler, and D. Stehlé, “CRYSTALS-Dilithium: A lattice-based digital signature scheme,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2018, no. 1, pp. 238–268, 2018.
- [12] T. Pornin and T. Prest, “More efficient algorithms for the NTRU key generation using the field norm,” in *Public-Key Cryptography — PKC 2019*, vol. 11443 of *Lecture Notes in Computer Science*, pp. 504–533, Springer, 2019.
- [13] P.-A. Fouque, J. Hoffstein, P. Kirchner, V. Lyubashevsky, T. Pornin, T. Prest, T. Ricosset, G. Seiler, W. Whyte, and Z. Zhang, “Falcon: Fast-Fourier lattice-based compact signatures over NTRU — specification,” NIST Post-Quantum Cryptography Standardization Submission, 2020. Version 1.2, <https://falcon-sign.info/>.
- [14] E. Barker and A. Roginsky, “Transitioning the use of cryptographic algorithms and key lengths,” *NIST Special Publication 800-131A*, 2019. Revision 2.
- [15] N. Bindel, U. Herath, M. McKague, and D. Stebila, “Transitioning to a quantum-resistant public key infrastructure,” in *Post-Quantum Cryptography (PQCrypto 2017)*, vol. 10346 of *Lecture Notes in Computer Science*, (Cham), pp. 384–405, Springer, 2017.
- [16] D. Stebila, S. Fluhrer, and S. Gueron, “Hybrid key exchange in TLS 1.3,” Internet Draft draft-ietf-tls-hybrid-design, Internet Engineering Task Force, 2023. Work in progress, <https://datatracker.ietf.org/doc/draft-ietf-tls-hybrid-design/>.
- [17] K. Kwiatkowski, P. Kampanakis, B. Westerbaan, and D. Stebila, “Post-quantum hybrid ECDHE-MLKEM key agreement for TLSv1.3,” Internet Draft draft-kwiatkowski-tls-ecdhe-mlkem, Internet Engineering Task Force, 2024. Work in progress; defines the X25519MLKEM768 TLS named-group codepoint (0x11EC). <https://datatracker.ietf.org/doc/draft-kwiatkowski-tls-ecdhe-mlkem/>.
- [18] P. Ravi, A. Chattopadhyay, J. Howe, A. Baksi, and S. Bhasin, “Side-channel and fault-injection attacks over lattice-based post-quantum schemes (CRYSTALS-dilithium and Falcon): Survey and new results,” *ACM Transactions on Embedded Computing Systems*, vol. 23, no. 2, pp. 1–54, 2024.
- [19] P. Pessl, L. G. Bruinderink, and Y. Yarom, “To BLISS-B or not to be: Attacking strongSWAN’s implementation of BLISS-B,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2019, no. 3, pp. 178–204, 2019.
- [20] C. Paquin, D. Stebila, and G. Tamvada, “Benchmarking post-quantum cryptography in TLS,” in *Post-Quantum Cryptography (PQCrypto 2020)*, vol. 12100 of *Lecture Notes in Computer Science*, (Cham), pp. 72–91, Springer, 2020.
- [21] D. Sikeridis, P. Kampanakis, and M. Devetsikiotis, “Post-quantum authentication in TLS 1.3: A performance study,” in *27th Annual Network and Distributed System Security Symposium (NDSS 2020)*, (San Diego, CA, USA), The Internet Society, 2020.
- [22] D. Stebila and M. Mosca, “Post-quantum key exchange for the internet and the open quantum safe project,” in *Selected Areas in Cryptography (SAC 2016)*, vol. 10532 of *Lecture Notes in Computer Science*, (Cham), pp. 14–37, Springer, 2017.
- [23] E. Androuraki, A. Barger, V. Bortnikov, C. Cachin, K. Christidis, A. De Caro, D. Enyeart, C. Ferris, G. Laventman, Y. Manevich, S. Muralidharan, C. Murthy, B. Nguyen, M. Sethi, G. Singh, K. Smith, A. Sornioti, C. Stathakopoulou, M. Vukolić, S. W. Cocco, and J. Yellick, “Hyperledger fabric: A distributed operating system for permissioned blockchains,” in *Proceedings of the Thirteenth EuroSys Conference*, (Porto, Portugal), pp. 1–15, ACM, 2018.
- [24] A. Baliga, N. Solanki, S. Verekar, A. Pednekar, P. Kamat, and S. Chatterjee, “Performance characterization of hyperledger fabric,” in *2018 Crypto Valley Conference on Blockchain Technology (CVCBT)*, (Zug, Switzerland), pp. 65–74, IEEE, 2018.
- [25] M. Kuzlu, M. Pipattanasomporn, L. Gurses, and S. Rahman, “Performance analysis of a hyperledger fabric blockchain framework: Throughput, latency and scalability,” *IEEE Access*, vol. 9, pp. 30887–30894, 2021.
- [26] Q. Nasir, I. A. Qureshi, and A. N. Jabbar, “Performance analysis of hyperledger fabric platforms,” in *Proceedings of the International Conference on Security and Management (SAM)*, CSREA Press, 2018.
- [27] P. Thakkar, S. Nathan, and B. Viswanathan, “Performance benchmarking and optimizing hyperledger fabric blockchain platform,” in *Proceedings of the 26th International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS)*, pp. 264–276, IEEE, 2018.
- [28] R. E. Campbell, “Transitioning to a hyperledger fabric hybrid quantum resistant-classical public key infrastructure,” *The Journal of The British Blockchain Association*, vol. 2, no. 2, pp. 1–11, 2019.
- [29] R. E. Campbell, “Evaluation of post-quantum distributed ledger cryptography,” *The Journal of The British Blockchain Association*, vol. 2, no. 1, pp. 17–24, 2019.
- [30] S. Brotsis, N. Kolokotronis, K. Limniotis, G. Bendiab, and S. Shiaeles, “On the security and privacy of Hyperledger Fabric: Challenges and open issues,” in *Proceedings of the IEEE World Congress on Services (SERVICES)*, (Beijing, China), pp. 197–204, IEEE, 2020.
- [31] J. Lee, “Towards quantum-safe smart contracts on Hyperledger Fabric,” master’s thesis, Delft University of Technology, August 2025.
- [32] P. Das, C. Jain, A. M. AlShahrani, M. Singh, M. Zeyauallah, and H. Hummad, “Quantum-resilient blockchain framework for privacy-preserving genomic data sharing and analysis,” *Journal of Information Security and Applications*, vol. 95, p. 104245, 2025.
- [33] K. Revathi and K. Suganthi, “Enhancing blockchain security against quantum threats through integration of post-quantum cryptographic algorithms,” *Computers and Electrical Engineering*, vol. 127, p. 110610, 2025.
- [34] D. Cooper, S. Santesson, S. Farrell, S. Boeyen, R. Housley, and T. Polk, “Internet X.509 public key infrastructure certificate and certificate revocation list (CRL) profile,” RFC 5280, Internet Engineering Task Force, 2008.
- [35] M. Ounsworth, J. Gray, M. Pala, and J. Klausner, “Composite ML-DSA for use in X.509 public key infrastructure,” Internet Draft draft-ietf-lamps-pq-composite-sigs, Internet Engineering Task Force, 2024. Work in progress, <https://datatracker.ietf.org/doc/draft-ietf-lamps-pq-composite-sigs/>.
- [36] Hyperledger Caliper Contributors, “Hyperledger caliper: A blockchain performance benchmark framework.” <https://github.com/hyperledger/caliper>, 2023. Version 0.6.0.
- [37] M. Mosca, “Cybersecurity in an era with quantum computers: Will we be ready?,” *IEEE Security & Privacy*, vol. 16, no. 5, pp. 38–41, 2018.
- [38] D. Moody, R. Perlner, A. Regenscheid, A. Robinson, and D. Cooper, “Transition to Post-Quantum Cryptography Standards,” Tech. Rep. NIST IR 8547 (Initial Public Draft), National Institute of Standards and Technology, Gaithersburg, MD, Nov. 2024.
- [39] National Security Agency, “Commercial National Security Algorithm Suite 2.0 (CNSA 2.0): Cybersecurity Advisory,” U.S. National Security Agency, Cybersecurity Advisory CSA-CNSA-2.0, 2022. Updated 2024; mandates full transition to quantum-resistant algorithms across U.S. national security systems by 2035.
- [40] Cyber Security Brunei, “Brunei Post-Quantum Cryptography Migration Framework,” Draft Version 0.1, Cyber Security Brunei, BruCERT, National Digital Forensics Laboratory, and Cyberwatch Centre, June 2026. National framework adopting NIST FIPS 203/204/205 with FN-DSA (FIPS 206 draft) and HQC for code-based diversity; targets RSA/ECC deprecation by 2030 and full migration by 2035; specifies a five-phase Assess–Select–Validate–Deploy–Monitor strategy.